Lab 02: Modelling Environmental Systems in Python

Gideon Bickler (470442980)

2026-08-09

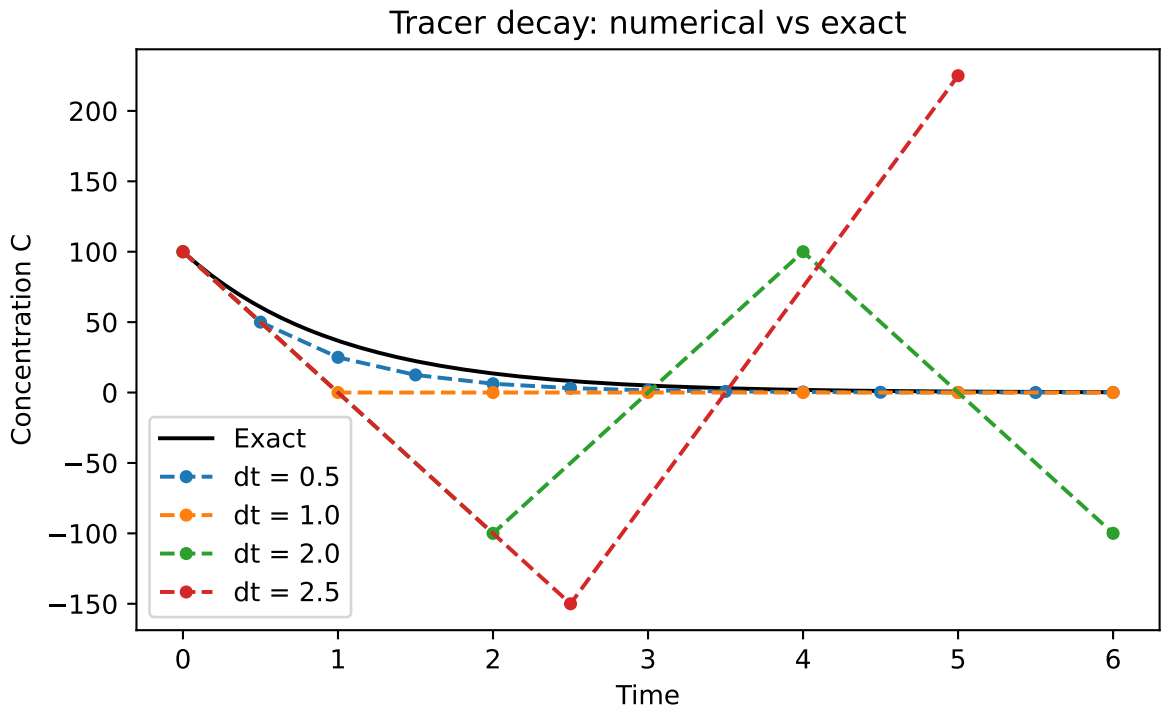
Setup

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

Part A: Tracer decay and step size (numerical)

```
def decay_numerical(C0, k, dt, t_end):
    """Return arrays of time and concentration for forward-Euler tracer decay."""
    n = int(t_end / dt)
    C = C0
    ts = [0.0]
    Cs = [C0]
    for i in range(n):
        C = C + (-k * C) * dt          # forward difference
        ts.append((i + 1) * dt)
        Cs.append(C)
    return np.array(ts), np.array(Cs)

# The exact answer, for comparison
C0, k, t_end = 100.0, 1.0, 6.0
t_fine = np.linspace(0, t_end, 200)
exact = C0 * np.exp(-k * t_fine)

fig, ax = plt.subplots(figsize=(7, 4))
```

```
ax.plot(t_fine, exact, "k-", label="Exact")

for dt in [0.5, 1.0, 2.0, 2.5]:
    ts, Cs = decay_numerical(C0, k, dt, t_end)
    ax.plot(ts, Cs, "o--", markersize=4, label=f"dt = {dt}")

ax.set_xlabel("Time")
ax.set_ylabel("Concentration C")
ax.set_title("Tracer decay: numerical vs exact")
ax.legend()
plt.show()
```



Answer to task A1: *(one to three sentences)*

The decay rate k controls the gradient of the slope.

Answer to task A2: *(one to three sentences)* Deviation from the exact value starts to be visible at dt = 2.0, becoming obvious at 5.0.

Answer to task A3: *(one to three sentences)* dt = 1.0 is the largest non-negative curve - above that the points overshoot. **Answer to task A4:** *(one to three sentences)* When the step size dt is too high, the model becomes unstable. When dt exceeds 1.0, the "updating"

section of the update rule (k.Cold.dt) becomes larger than Cold itself when k is also 1, causing it to overshoot into the negatives (and then way too far into the positives once Cold becomes negative and so on) resulting in oscillations.

Part B: Reservoir box model (process-based)

```
import numpy as np
import matplotlib.pyplot as plt

# ---- Parameters ----
V = 1.0e6        # lake volume (m^3)
Q = 10.0e3        # river inflow (m^3 per day)
C_in = 10.0      # incoming concentration (mg/L)
k = [0.02, 0.05, 0.1, 0.2]       # in-lake decay rate (per day)

# ---- Time settings ----
dt = 1.0         # time step (days)
days = 200

# ---- Initial condition ----
C = 20        # polluted lake (from part 2, unused in 3)
# ---- Storage ----
history = [C]
results = {}
# ---- Time loop ----
for k_values in k:
    C = 0                # reset initial condition for each k
    history = [C]         # reset history for each k
    for _ in range(days):
        dCdt = (Q * C_in - Q * C - k_values * V * C) / V
        C = C + dCdt * dt
        history.append(C)
    results[k_values] = history

# ---- A quick look at the numbers ----
for k_values, history in results.items():
    print(f"Started at {history[0]:.2f} mg/L")
    print(f"After 10 days: {history[10]:.2f} mg/L")
    print(f"After 50 days: {history[50]:.2f} mg/L")
    print(f"After {days} days: {history[-1]:.2f} mg/L")
```

```
Started at 0.00 mg/L
After 10 days: 0.88 mg/L
After 50 days: 2.61 mg/L
After 200 days: 3.33 mg/L
Started at 0.00 mg/L
After 10 days: 0.77 mg/L
After 50 days: 1.59 mg/L
After 200 days: 1.67 mg/L
Started at 0.00 mg/L
After 10 days: 0.63 mg/L
After 50 days: 0.91 mg/L
After 200 days: 0.91 mg/L
Started at 0.00 mg/L
After 10 days: 0.43 mg/L
After 50 days: 0.48 mg/L
After 200 days: 0.48 mg/L
```

Answer to task B1: *(short answer)* Analytical Steady State: 0.91mg/L (2.d.p) (or 0.9090909…) State of the numerical model after 200 days: 0.91mg/L The numerical model is in fact therefore reaching the analytical steady state.

Answer to task B2: *one sentence* The resevoir reaches the same predicted steady state after 200 days.

Answer to task B3: *one sentence* The steady state rises linearly with the size of Q, as can be seen in the steady state equation as Q*Cin increases proportionally more than Q + kV due to V being very large.

Answer to task B4: *one sentence* As k increases, the steady state concentration decreases and the approach gets more gradual.

Reflection

The greatest difficulty I had was remembering how both to use github - as I had previously only used it for Java, as we used a slighly different method (I believe using git bash although my memory is foggy). It was also a struggle learning how to use python for loops as they work quite differently (although more easily once I learned them) to those in other languages I have used. For part B if I had more time I would use a wider range of days and plot the sweep of k values together to make the differences more obvious.

Checklist before you push

- ☐ Every task has a code cell **and** a short prose answer
- ☐ Every figure has axis labels and a title
- ☐ Your name and student ID are in the YAML header
- ☐ The file renders cleanly (no red error boxes in the HTML output)
- ☐ Filename is `lab-XX-topic.qmd`, not `Untitled.qmd`